

Application Program Development

Ch 02: Input, Calculations and Formatted Output

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Reading Input from the Keyboard

- Most programs need to read input from the user
- Built-in `input()` functions reads input from standard input (keyboard)
 - Returns the data as a string
 - Does not automatically display a space after the prompt

```
>>> name = input('What is your name? ')
What is your name? Alan Turing
>>> print(name)
Alan Turing
```



Reading Numbers with the `input()` Function

- `input()` function always returns a string
- Built-in functions convert between data types
 - `int(item)` converts *item* to an int
 - `float(item)` converts *item* to a float
 - **Nested function call:** are allowed, output from inner function becomes parameter input to outer function.
 - Type conversion only works if item is a valid numeric literal, otherwise throws exception

```
>>> hours = input('How many hours did you work? ')
How many hours did you work? 22
>>> type(hours)
<class 'str'>
>>> hours = int(hours)
>>> type(hours)
<class 'int'>
>>> print(hours)
22
>>> hours = int(input('How many hours did you work? '))
How many hours did you work? 33
>>> type(hours)
<class 'int'>
```



Performing Calculations

- Math expression: perform calculation and gives a value
 - **Math operator**: tool for performing calculation
 - **Operands**: values operations are applied to
 - Variables can be used as operands
 - Resulting value typically assigned to a variable
- **Note**: Two types of division:
 - / operator performs floating point division
 - // operator performs integer division

Symbol	Operation	Description
+	Addition	Adds two numbers
-	Subtraction	Subtracts one number from another
*	Multiplication	Multiplies one number by another
/	Division	Divides one number by another and gives the result as a floating-point number
//	Integer division	Divides one number by another and gives the result as a whole number
%	Remainder	Divides one number by another and gives the remainder
**	Exponent	Raises a number to a power



Operator Precedence and Grouping with Parentheses

Python Operator Precedence

- ① Operations enclosed in parentheses
 - Forces operations to be performed before others
 - ② Exponentiation **
 - ③ Multiplication *, division / and //, and remainder %
 - ④ Addition + and subtraction -
-
- Higher precedence performed first
 - Same precedence operators execute from left to right



Converting Math Formulas to Programming Statements

- Operators required for any mathematical operation
- When converting mathematical expression to programming statement:
 - May need to add multiplication operators
 - May need to insert parentheses

Algebraic Expression	Python Statement
$y = 3\frac{x}{2}$	<code>y = 3 * x / 2</code>
$z = 3bc + 4$	<code>z = 3 * b * c + 4</code>
$a = \frac{x+2}{b-1}$	<code>a = (x + 2) / (b - 1)</code>
$y = 3x^2 + 4x - 5$	<code>y = 3 * x**2 + 4 * x - 5</code>



Mixed-Type Expressions and Data Type Conversion

Data type resulting from math operation depends on **data types** of operands

- Two int values: result is an int
- Two float values: result is a float
- int and float: int converted to float, result of operation is float
- Type conversion of float to int causes truncation of fractional part



Breaking Long Statements into Multiple Lines

- Long statements cannot be viewed on screen without scrolling and cannot be printed without cutting off (see [PEP8 Python Formatting Standards](#))
- Multiline continuation character \: Allows to break a statement into multiple lines (PEP8 recommends breaking before operator for readability)

```
result =    var1 * 2 + var2 * 3 \  
          + var3 * 4 + var4 * 5
```

- Any part of a statement that is enclosed in parentheses can be broken without the line continuation character

```
result = (  var1 * 2 + var2 * 3  
          + var3 * 4 + var4 * 5)
```

```
print("Monday's sales are", monday,  
      "and Tuesday's sales are", tuesday,  
      "and Wednesday's sales are", wednesday)
```



String Concatenation and Repetition

You can use string concatenation to combine strings

```
my_str = 'Hello' + 'world'
```

The * operator performs string repetition

```
>>> my_str = 'Hello ' + 'World '
```

```
>>> print(my_str)
```

```
Hello World
```

```
>>> big_hello = my_str * 3
```

```
>>> print(big_hello)
```

```
Hello World Hello World Hello World
```



More About the print() Function

- print() function displays line of output
 - Newline character at end of printed data
 - Special argument end='delimiter' causes print() to place *delimiter* at end of data instead of newline character
- print() function uses space as item separator
 - Special argument sep='delimiter' causes print() to use *delimiter* as item separator
- Special characters appearing in string literal (see [Escape Sequences in C](#))
 - Preceded by backslash \ul> - Examples: newline \n horizontal tab \t
- Treated as commands embedded in string



Displaying Formatted Output with F-strings (1 of 5)

- An **f-string** is a special type of string literal in Python that is prefixed with the letter **f**
- **F-strings** support placeholders for variables

```
>>> name = 'John VonNeumann'  
>>> print(f'Hello {name}!')  
Hello John VonNeumann!
```

- Placeholders can be expressions that are evaluated

```
>>> print(f'The value is {10 + 2}.')  
The value is 12.  
>>> val = 10  
>>> print(f'The value is {val + 2}.')  
The value is 12.
```



Displaying Formatted Output with F-strings (2 of 5)

- Format specifiers can be used with placeholders

```
>>> print(f'{num:.2f}')  
123.46
```

- .2f means:
 - round the value to 2 decimal places
 - display the values as a floating-point number

See [PEP 3101 – Advanced String Formatting](#) for a full list of all format specifiers



Displaying Formatted Output with F-strings (3 of 5)

Specifying a minimum field width:

```
>>> num = 12345.6789
>>> print(f'The number is {num:12,.2f}')
The number is      12,345.68
```

The number is

			1	2	,	3	4	5	.	6	8
--	--	--	---	---	---	---	---	---	---	---	---

└────────────────┘
 ↑
Field width = 12



Displaying Formatted Output with F-strings (4 of 5)

- Aligning values within a field
 - use < for left alignment
 - Use > for right alignment
 - Use ^ for center alignment

```
>>> name = 'Alan Turing'
>>> title = 'Computer Scientist'
>>> salary = 4567.1234
>>> print(f'{name:a<20s}|{title:b^40s}|{salary:c>20.2f}')
Alan Turingaaaaaaaaa|bbbbbbbbbbbComputer Scientistbbbbbbbbbbb|cccccccccccc4567.12
```



Displaying Formatted Output with F-strings (5 of 5)

- The order of designators in a format specifier
 - When using multiple designators in a format specifier, write them in this order:

[alignment] [width] [,] [.precision] [type]

- Example:

```
print(f'{number:^10,.2f}')
```



Magic Numbers

- A magic number is an unexplained numeric value that appears in a program's code.

```
amount = balance * 0.069
```

- What is the value 0.069? An interest rate? A fee percentage? Only person who wrote the code knows for sure (bad communication).
 - Can be difficult to determine its purpose
 - If it needs to be used in multiple places, can lead to bugs if needs to be changed.
 - You might make a mistake each time you retype in the magic number.



Named Constants

- **Use named constants instead of magic numbers**
- A named constant is a name that represents a value that does not change during the program's execution.

```
INTEREST_RATE = 0.069
```

- [Python PEP8 style](#) states named constants should use UPPER_CASE_SNAKE_VARIABLES

Advantages of Using Named Constants

- Named constants make code self-explanatory, self-documenting, readable code
- Named constants make code easier to maintain
- Named constants help prevent typographical errors and bugs



Summary

- Use the Python `input()` function to read input from standard in (keyboard)
 - Always returns a `str` string type
- Use `int()` and `float()` functions to convert string to a numeric value
 - Can *nest* function calls, which is common when needing numeric input
- Be aware of the difference between integer and floating-point division
- Understand operator precedence, parentheses `()` always have highest precedence
- There are differences between standard mathematical notation and how you write a mathematical expression in Python
- Learn and use formatted f-strings in Python for more control of output
 - Can round floats to number of desired decimal places
 - Can set field width of strings and floats
 - Can left, center, and right justify output within field
 - and much more



References



EAST TEXAS A&M